

Design Strategies for Dynamic Single-Event Effect Testing of Algorithmic Computation

I. Hudson, *Member, IEEE*, H. Hunnicutt, *Member, IEEE*, L. Raymond, *Member, IEEE*, K. Lingasubramanian, *Member, IEEE*, and T. D. Loveless, *Senior Member, IEEE*

Abstract—Dynamic single-event testing must run an application-representative work cycle under an accelerated flux that interrupts it, and every interruption costs dead time, in which the host cannot observe the device. To compare design strategies, eight firmware builds running a digital filter on the MSP430FR6989 microcontroller were tested under heavy ions, varying data storage (SRAM or FRAM), clock frequency, and work-cycle length. Some of these choices, including data storage, moved the corruption-event cross section, which counts errors seen at the output of computation from one or more single-event upsets (SEUs), by a factor of 9.5. Others, such as batch size, which largely sets the work cycle, had no detectable effect, indicating that a test can adjust its work cycle while its code stays representative of the mission. The functional-interrupt cross section, which counts resets caused by either a single-event functional interrupt (SEFI) or multiple SEUs overwhelming correction logic, varied by only a factor of 1.4. An existing flux-selection method, built to reveal a rare failure before a dominant one, was tested against these data with the work cycle varied through the firmware. One cross section for interrupting a running cycle described all six compared builds across work cycles from 20 to 814 ms. Flux and work cycle can therefore be calibrated together for a target share of cycles that complete without an interrupt. Extending the method to dead time shows that how the host detects functional interrupts and recovers from them determines how much beam time is productive.

Index Terms—dead time, dynamic testing, ferroelectric RAM (FRAM), firmware, flux selection, heavy ions, microcontrollers, MSP430, observability, recovery time, single-event functional interrupt (SEFI), single-event upset (SEU), test-as-you-fly, work cycle

I. INTRODUCTION

THE processing of data through complex microelectronic systems occurs at rates that present a fundamental challenge to radiation effects characterization. Even in low-power microcontroller units (MCUs), a finite impulse response (FIR) filter propagates data through many functional hardware blocks millions of times per second. Static and semi-static interrogation offers experimental simplicity and reproducibility, but it does not capture the fault modes accessible only during active processing. Dynamic testing, in which the device remains

active under irradiation, is required to characterize corruptions of computational results and control flow that static states cannot reveal [1, 2].

During dynamic testing, a device's internal hardware blocks will move in and out of operation based on the exact programming choices the designer made. For a complex commercial off-the-shelf (COTS) microcontroller, it is likely the designer will have a choice among several internal modules capable of accomplishing the same task. These modules respond differently to a harsh environment, so the sensitivity of a COTS microcontroller to single-event effects (SEEs) depends on which internal hardware blocks the running code exercises [3–7]. A test program that exercises only the programmer-visible registers can overstate a processor's upset rate by up to an order of magnitude against application-like code [8]. Test-as-you-fly procedures aim to compensate for these application-specific challenges by using the true device operating state in flight, including both representative programming and inputs [1, 9].

This representation challenge is compounded by the accelerated flux needed for statistical confidence in a fixed test time. The timing of a work cycle and the device's susceptibility to functional interrupts together create a tension in an accelerated SEE experiment. If the work cycle is long and the susceptibility is high, a high flux drives the device into functional interrupt faster than the dynamic process can be observed. To plan around this, an experimenter should know how susceptible the device is to these interrupts, which has been measured under protons for several microcontrollers and a Linux-class system-on-chip [10, 11]. The experimenter should also be able to gauge a range of flux values that will allow delivery of a high share of work to the host without frequent interruptions. The work completed between interrupts has been reported as a mean work to single-event functional interrupt (SEFI), used to compare processor configurations [5, 11], following the mean workload between failures introduced for GPU reliability [12]. The same tradeoff appears on the MSP430FR5969, where software redundancy lowered the program-error cross section by a factor of 17 to 29 but slowed execution enough that the work completed between failures rose only by a factor of 4 to 7 [13].

Although many classes of devices will be susceptible to functional interrupts due to corruption of essential memory structures, each differs in how, and how quickly, it recovers. Many microcontrollers that boot from on-chip non-volatile memory reset themselves in milliseconds when an internal reset path is available [2, 14, 15], while systems-on-chip that run Linux, for example, are recovered by a host-commanded

This work was supported in part by the Defense Threat Reduction Agency under contract HDTRA12020002 and the Missile Defense Agency under contract HQ0860-23-C-7120. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Government.

I. Hudson, H. Hunnicutt, and T. D. Loveless are with the IU Center for Reliable and Trusted Electronics (IU CREATE), 2401 N. Milo B. Sampson Lane, Bloomington, IN, 47408, USA; Email: isahudso@iu.edu

L. Raymond and K. Lingasubramanian are with the CFD Research Corporation, 6820 Moquin Drive NW, Huntsville, AL 35806, USA.

reboot or power cycle taking tens of seconds to minutes [11, 16]. Masking of interrupts at high flux has been reported where recovery took seconds to a minute [15].

Established flux-selection guidance addresses several problems, including bounding flux to avoid measurement artifacts [1, 17, 18] or overwhelming correction logic [18–21]. Those guidelines target measurement artifacts. The observability of a device executing dynamic work cycles while susceptible to functional interrupts is a separate question. The European Space Components Coordination (ESCC) No. 25100 specification asks that functional interrupts be characterized in all of a device’s operating conditions and ties the flux to that characterization qualitatively [22]. It requires that the share of test time in which a fault could be detected be measured and applied to the cross section, and it asks that flux keep the remaining dead time, in which a fault could not be detected, under 20% of that share. Zimmaro et al. [23] established a method tying flux to observability, choosing flux to maximize the probability of observing a low-probability failure before a dominant one interrupts the work cycle. It was developed for system-level testing under protons and mixed fields, where the system fixes its full working cycle. The cross section of its main failure, the most probable one, then sets how many cycles complete without that failure. This work puts the same survival relation to a new use, calibrating flux against work cycle when the test designer can choose the work cycle. It builds on the method by testing the relation across intentionally varied work cycles, checking it against a direct count of surviving cycles, and adding the dead time each interrupt costs.

This paper examines interrogation strategies for the MSP430FR6989, a low-power mixed-signal microcontroller made by Texas Instruments (TI), during algorithmic computation of a low-pass FIR filter. After Section II describes the campaign and how its events are counted, the paper makes three contributions. Section III shows that the device’s own reset path allows the beam to remain on for most functional interrupts, and that firmware choices shift the corruption-event cross section by a factor of 9.5 while the functional-interrupt cross section shifts by a factor of 1.4. Section IV-A tests that survival relation [23] against a direct count of surviving work cycles ranging from 20 to 814 ms and turns it into a calibration of flux against work cycle. Section IV-B extends that relation to dead time, the time the host spends detecting and recovering from an interrupt, and measures how often a second strike during it went undetected.

II. EXPERIMENTAL DESIGN

The experimental conditions are described in Table II, including the reference condition, the linear energy transfer (LET) and flux at which every firmware build was run for comparison. The surface LET is reported throughout.

A. Hardware Setup

Each device under test (DUT) was installed on an MSP-TS430PZ100D target board mounted to a custom routing board, and the six comparison builds of Section III-B ran on two of the eleven devices, marked A and B in Table I.

A ribbon cable carried power, inter-integrated circuit (I2C) communication, and status signals to a TUL PYNQ™-Z2 test host outside the beam, which managed each test run and logged results, and is referred to throughout as the host. An MSP-FET debug probe provided programming, reset, and power monitoring over the Joint Test Action Group (JTAG) interface. The MSP430FR6989 contains ferroelectric random-access memory (FRAM) protected by error-correcting code (ECC), 2 kB of SRAM, and a hardware multiplier [25]. Per TI documentation, the FRAM error-correction hardware corrects single-bit errors and separately flags, but does not correct, multi-bit errors [26].

B. Experimental Work Cycle and Firmware Builds

The basic work cycle defined for the DUT began with the reception of a 64-sample array of 16-bit Q15 fixed-point numbers from the host over I2C. The array was stored in either SRAM or FRAM depending on the firmware variant, and Table I gives full details on the eight variants. The DUT then ran TI’s MSP DSP Library (DSPLib) [27] 16-tap Q15 FIR function, storing the result in FRAM for every variant except *SRAM_B1*. Batch size B set how many sequential computations the DUT performed on one stored input and was varied to increase how much of each work cycle was spent computing rather than communicating. A B of 1 transmits each result immediately, while a B of 50 computes 50 results from the same input before transmitting them together. *SRAM_Mixed_B200* placed all control variables into SRAM, but required FRAM for output storage because of the output structure’s size.

Another method employed to increase the share of each work cycle spent computing was slowing, or down-throttling, the clock frequency used during computation. *FRAM_B1_Throttled* and *FRAM_B200_Throttled* slowed to 1 MHz from 16 MHz. *FRAM_B200_Slow* instead slowed the clock frequency throughout to 8 MHz in an attempt to isolate frequency-related effects. The FRAM controller requires wait states for any clock above 8 MHz, as the hardware cannot reliably be read or written at higher frequencies. *FRAM_B1_Throttled* and *FRAM_B200_Throttled* cross that 8 MHz boundary twice per cycle, between a 16 MHz communications clock and a 1 MHz compute clock.

Table I also gives the share of each work cycle spent in computation and the duration W of the whole work cycle, from input reception to output transmission. This duration is what Zimmaro et al. [23] call the full working cycle time, the minimum time a system needs to complete all of its operations. It is measured here from event-free runs, meaning runs with no recovery event of the kinds defined in Section II-C. The work-cycle time W is a mean and the computation share is a median.

C. Functional Interrupt Recovery

Each run of the campaign is divided into calls, which describe one uninterrupted stretch of host operation from the startup handshake to the point where the host stopped, whether by reaching its planned end or by the device requiring manual

TABLE I
FIRMWARE BUILDS AND MEASURED WORK-CYCLE TIMING

Firmware	Device	Storage	Batch B	f_{comp} (MHz)	f_{comms} (MHz)	Compute share (% of W)	W (ms)
FRAM_B1_Throttled	B	FRAM	1	1	16	60.5	45
FRAM_B1	A	FRAM	1	16	16	9.3	20
SRAM_B1	B	SRAM	1	16	16	8.7	20
FRAM_B50	A	FRAM	50	16	16	48.3	216
SRAM_Mixed_B200	B	SRAM (output FRAM)	200	16	16	48.5	789
FRAM_B200	A	FRAM	200	16	16	50.5	814
FRAM_B200_Throttled [†]	B	FRAM	200	1	16	93.2	5969
FRAM_B200_Slow [†]	C	FRAM	200	8	8	52.1	1560

f_{comp} and f_{comms} are the clock frequencies during computation and communication. Device is the device each build ran on at the reference condition.

[†] Excluded from the comparison of Section III-B.

TABLE II
EXPERIMENTAL CONDITIONS

Parameter	Value
Facility	K500 cyclotron, Facility for Rare Isotope Beams (FRIB), Michigan State University [24]
Beam	19.5 MeV/u cocktail of Ne, Ar, Co, Kr, Ag, and Tm, with aluminum degraders
Surface LET (MeV cm ² /mg)	Ne 2.3, 4.1 Ar 7.9, 9.4, 10.9 Co 15.7 Kr 27.9, 32.3 Ag 45.6 Tm 71.7
Range in silicon	95 to 370 μm across the cocktail
Incidence	Normal
Flux	1.1×10^3 to 9.4×10^5 cm ⁻² s ⁻¹
Devices	Eleven MSP430FR6989 microcontroller units, delidded by chemical etching
Supply and temperature	3.3 V, ambient
Reference condition	Surface LET 7.9 MeV cm ² /mg (Ar) and flux 4.6 to 5.1×10^4 cm ⁻² s ⁻¹

functional-interrupt mitigation. Within a call, the DUT ran consecutive work cycles. A run's exposure and its counted events were accumulated call by call. A run itself ended on reaching a targeted fluence, on a count of 10 functional interrupts requiring manual mitigation, or on losing the device to a functional interrupt requiring a device change or a reprogram. Every firmware build also reported a boot identifier and a startup handshake upon restart. The host used these to confirm that a reset had occurred and to keep a running count, even when it missed the event at the time.

Recovery began on any of three detection signals. The first was a timeout, meaning no response within the allotted time to a transmission, an expected reply, or a completed computation. Another was a malformed payload, a reply that arrived in time but consisted entirely of the device's own startup handshake instead of real content. Finally, there was a data fault, a reply that arrived on time and mostly intact but carrying recognizable artifacts such as fill patterns or handshake bytes. Each timeout or malformed payload opened a recovery episode, which could hold several recovery attempts.

A timeout and a malformed payload both went straight into the same fully automatic recovery process, in which the host stopped issuing work and polled the device until its boot handshake reappeared. The resets behind these interrupts were self-triggered, and each ended, like every MSP430 reset, in a power-up clear (PUC), re-initializing most peripheral modules and registers to their reset values [26]. The DUT's own protection circuitry resets the device without any command from the host, in response to an uncorrectable FRAM error, a power-supply brownout, or a hardware fault it detects internally. After an automatic recovery, the host resynchronized and re-ran the interrupted work cycle from its input, a retried execution that Section IV-A does not count as a new attempted cycle. A data fault instead triggered one repeated iteration, with the input resent and the result recomputed, and only a data fault on that repeated iteration opened a recovery episode and went to automatic recovery. When the device did not return on its own, recovery escalated to a beam pause, a manual read of the entire memory space, a host-issued reset over JTAG, and, if needed, reprogramming or a power cycle.

D. Classifying Errors and Counting Events

This work reports two primary quantities, the *functional interrupt* and the *corruption event*, and names each by its effect rather than its cause. A single-event upset (SEU) and a SEFI are both defined by attribution to one ionizing particle [3, 28], and this campaign does not have the granularity to establish that. A functional interrupt recorded here is equally consistent with one strike on a register cell [29] and with several independent strikes overwhelming correction logic [18]. These are the quantities conventionally reported as SEFI and SEU counts, without the single-particle attribution those terms carry, and Table III defines each with the exposure basis by which it is divided.

Functional interrupts are counted three ways here, and each gives its own cross section. The reset count reads the DUT's boot identifier (Section II-C) and credits the difference between consecutive confirmed readings, so it includes interrupts the host missed. It gives σ_{FI} , the primary count of Section III, because the host is blind from a reset until it resynchronizes, the dead time of Section IV-B, and its own detections omit

the interrupts that land in that window. The host-detected count, giving σ_{det} , records what the host observed in real time (Section III-A). It is the basis for the recovery statistics and the beam-time accounting. The gap between these two counts is not all missed interrupts, since one event can also reset the device more than once, and Section IV-B separates the two. The third is a direct count of lost work cycles, giving σ_{cyc} and the clean-cycle fraction of Section IV-A. A cycle is lost once however many detections it carries, so this count sits below the host-detected one.

A *counted error* is read from a delivered output. Detection begins with a comparison between the received outputs and “golden outputs” captured before irradiation, but a raw mismatch count cannot separate a genuine computation error from an artifact of the transfer, the memory, or a reset. Two screening steps therefore reduce the raw mismatches to counted errors. The first removes known artifacts of an interrupt, such as fill patterns, meaning anomalously long runs of 0s or 1s, and repeated contamination from the I2C boot handshake. At the reference condition of Table II, this step removes 25,473 of 27,188 raw mismatches and leaves 1,715 outputs.

The second step asks whether a recovery was started during the work cycle that delivered the output. If it was, the output is removed whatever its content, because nothing in the record separates a genuine computation error inside that cycle from a consequence of the disturbance. The corruption-event exposure of Section III-B does not credit that cycle either, so the count and the exposure by which it is divided cover the same cycles. A repeated iteration that started no recovery does not remove the output. At the reference condition, this step removes 1,082 outputs, leaving 633.

Each counted error is then tested against the simplest physical explanation, a single bit flip in the computation that produced it. Flipping one bit at a time in simulation, in the input sample, the filter position, the filter coefficient, the accumulator, or the output sample, shows whether any single flip reproduces the returned output exactly, in the manner of fault-injection studies [30]. The test locates faults for Section III-B and removes nothing, so all 633 remain counted errors. No flip in a filter coefficient or in the filter position reproduced any of them.

Finally, contiguous counted errors within one call are collapsed into a single corruption event, so that one persistent fault, such as a corrupted input sample reused by every later computation in a batch, is not counted as many separate events. The count is therefore a lower bound, since several independent strikes within one run of errors would be counted once. Of the 633 counted errors, 549 fall in such runs and collapse to 11 corruption events, while the remaining 84 stand alone as single corrupted outputs. This leaves 95 corruption events across the six builds at the reference condition.

Each reported rate is a count divided by the fluence Φ accumulated on its own exposure basis, the flux ϕ integrated over the time in that basis (Table III), in cm^2 per device. The detection choices of Section II-C and the screening choices of this section are this campaign’s own. Each rate is therefore reported as an operational cross section, an observed rate

TABLE III
COUNTS, CROSS SECTIONS, AND EXPOSURE BASES

Count	Definition	Exposure basis
Functional interrupt, σ_{FI} (a SEFI or accumulated SEUs)	One increment of the device’s own reset counter, whether or not the host detected it, plus one for each escalated recovery that ended its call before the counter could be read again (16 at the reference condition, under 2% of the count)	Beam-on time while the host ran a call; no reset could be counted between calls
Host-detected interrupt, σ_{det}	One detection in the host’s recovery record, each opening one recovery episode	As above
Lost work cycle, σ_{cyc}	An attempted work cycle in which a host-detected interrupt occurred, counted once however many it carried; a retried execution is not an attempt	In-cycle time of attempted cycles
Counted error (one or more SEUs)	One corrupted result that passed both screening steps	Not normalized, since a persistent fault repeats it in later results
Corruption event (one or more SEUs)	One or more contiguous counted errors in a single call, counted as one fault and kept only if no recovery began in the cycle that delivered it	In-cycle time of attempted cycles, scaled by the clean-cycle fraction θ^*

* θ : the measured share of attempted work cycles in which no host-detected interrupt occurred (Section IV-A).

under those choices rather than a device cross section, after the component-level and system-level distinction drawn for system-level qualification [31].¹

III. EXPERIMENTAL RESULTS

Six of the eight firmware builds produced usable output at the reference condition. At that flux, *FRAM_B200_Throttled* could not deliver comparable output, consistent with combining a long interval between committing work to the host with a frequent interruption rate. *FRAM_B200_Slow* ran too little to be compared. Both were set aside early in the campaign, so their rates carry little confidence, and neither is used further.

A. Recovery Observations

Across the campaign, the host detected and acted on 4,623 interrupts, each opening one recovery episode, and every recovery was recorded with its trigger and its outcome. An interrupt counted from that record is the host-detected interrupt of Section II-D, which gives a lower count than the device’s own reset counter. Malformed payloads were by far the most common, with 3,817 observations against 769 timeouts and 37 data faults. That split is consistent with the firmware rearming its I2C interface on startup, so a reset device is far more likely to answer with the wrong content than to stay silent long enough to trigger a timeout.

¹Zero-count configurations use the one-sided 95% upper limit $2.996/\Phi$. Positive counts use two-sided exact 95% Garwood (Poisson) intervals [32, 33].

Recovery success was initially judged on the boot handshake alone. That proved not to confirm full application recovery, since the device was sometimes observed delivering a work cycle that did not match the golden output despite a successful handshake. Scored against the share of all work cycles that returned a corrupted result, this occurs 7.6 times more often than chance alone predicts (156 observed against an expected 20.4, Poisson $p \ll 0.001$), so the fault often outlived the reset, rather than a new strike corrupting the next cycle.

Success is therefore scored here against the next work cycle the device delivered. On that basis, the automatic path proved as reliable as a manual reset. Among recoveries the device later had an opportunity to prove, 3,172 of 3,328 were successfully recovered by the automatic path against 107 of 111 by the manual path, 95.3% against 96.4%. The difference is one percentage point, with a 95% interval of -5 to $+3$ points (Fisher exact, $p = 0.82$). The handshake had reported success on all 160 manual resets over JTAG. Of these, 49 could not be scored, because a further mitigation or the end of the run came before the device attempted a work cycle. An automatic recovery took a median of 0.165 s against 84.6 s for a manual reset with the beam paused, so the cost of recovery concentrates in escalation. Escalation depended on how the interrupt was detected. Without any manual step, the automatic sequence closed 99.3% of malformed payloads, 89.2% of data faults and only 80.4% of timeouts.

FRAM_B1_Throttled's dedicated sweep across the full tested LET range shows the value of the automatic recovery mechanism. Counted from the reset counter rather than from detections, this build produced 4,448 functional interrupts across the sweep, of which 4,292 (96.5%) closed automatically, 139 (3.1%) needed a manual reset, and 17 (0.4%) ended their run without recovering. However, the 3.5% not handled automatically consumed 2.6 hours of beam pauses, 57.5% of the 4.5 hours this build spent either under beam or paused for recovery. Had each detection of functional interrupt cost the median 84.6 s manual pause, that same test time would have delivered $1.45 \times 10^7 \text{ cm}^{-2}$ instead of the $2.56 \times 10^8 \text{ cm}^{-2}$ actually accumulated, a factor of 18 less exposure. Fig. 1 shows the same split holding at every LET tested, with automatic recovery ahead of manual recovery by roughly one to two orders of magnitude at most LETs and unrecovered interrupts rarer still or absent.

B. Firmware Comparison and Exposure Normalization

Fig. 2 compares the six firmware builds' responses at the reference condition. A functional interrupt is read from the device's own reset counter, which also records resets outside work cycles, so its rate normalizes by beam-on exposure. A corruption event has to be read off a delivered result, so its rate divides only by the fluence delivered during work cycles that returned a clean result. The numerator is the 95 corruption events of Section II-D. Crediting exposure from cycles that never delivered a result would artificially shrink the cross section for the builds that lose the most. The basis matters because only 24 to 43% of beam-on time in a call fell within

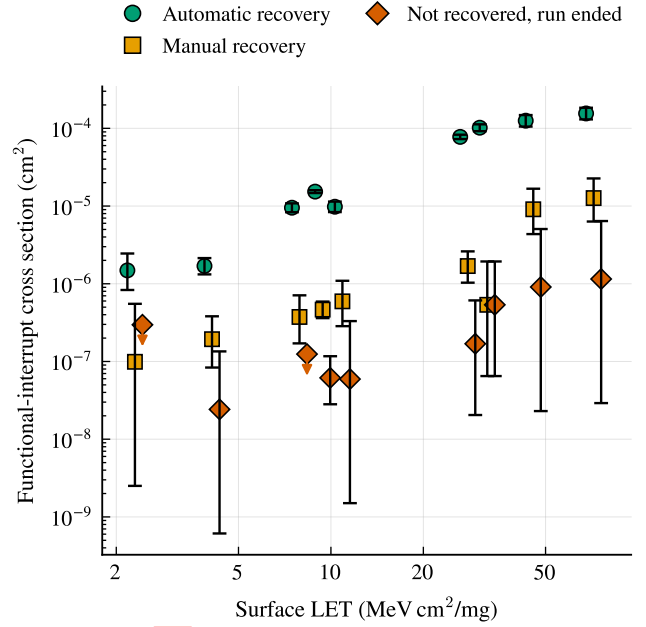


Fig. 1. *FRAM_B1_Throttled*'s functional-interrupt cross section by surface LET across its own 35-run sweep, split by how each functional interrupt was resolved. Counts are increments of the device's own reset counter over beam-on exposure, each credited to the resolution of the recovery episode in which the counter was next read. Error bars are 95% exact intervals and downward arrows are one-sided 95% upper limits where no interrupt of that kind occurred. Flux was matched to the expected event rate, so it falls from about $1 \times 10^5 \text{ cm}^{-2}\text{s}^{-1}$ at the lowest LETs to about 1×10^3 at the highest.

work cycles, and most of the remainder went to handling functional interrupts. Recovery took 14 to 36% and the host's wait to detect an interrupt up to 29%, together forming the dead time, in which the host could not observe the device. Re-running cycles after an interrupt or a data fault took 1 to 2% at $B = 1$ but about 40% at $B = 200$. Because a re-run does not begin from a verified state (Section III-A), it is not counted as a new work cycle.

The functional-interrupt rate for the five directly comparable builds, all but *SRAM_B1*, whose reset counter could not be reconstructed, ranges $7.7\text{--}11.1 \times 10^{-6} \text{ cm}^{-2}$, a factor of 1.4, and their count-weighted pooled rate is $\sigma_{\text{FI}} = 9.4 \times 10^{-6} \text{ cm}^{-2}$. Host detections alone over the same exposure vary by a factor of 1.2, with the reset count running 1.17 to 1.44 times the detected count depending on the build. Section IV-B separates the part of this gap made of missed interrupts from repeated resets that followed a single event.

Both SRAM builds held the reset counter in unprotected SRAM, where the same counter bit failed repeatedly after a reset, a retention problem rather than a series of independent strikes. The counter of *SRAM_B1* could not be reconstructed, so that build is excluded from the pooled functional-interrupt rate. The count for *SRAM_Mixed_B200* is kept after removing any possible overcount, which makes it a lower bound.

A Pearson chi-squared test for one shared functional-interrupt rate across the five comparable builds rejects it, $\chi^2 = 12.3$ on four degrees of freedom, $p = 0.016$. That rejection depends on how multi-reset gaps are credited (Section V). *FRAM_B50*, the lowest rate of the group, and

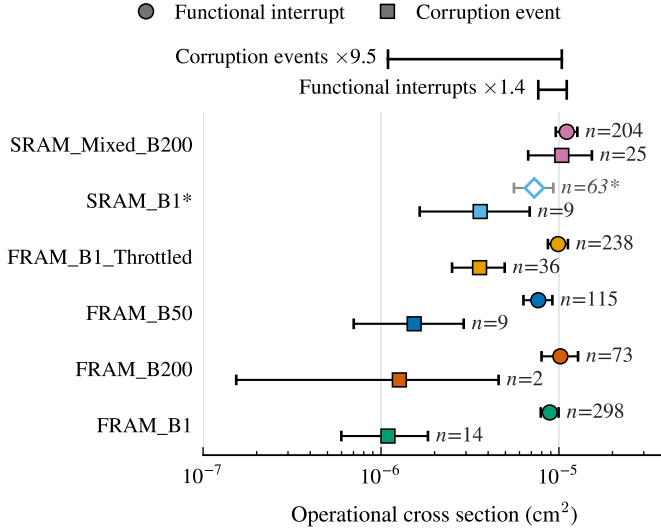


Fig. 2. Operational cross sections for six firmware builds at the reference condition, surface LET 7.9 MeV cm²/mg and flux $4.6\text{--}5.1 \times 10^4 \text{ cm}^{-2}\text{s}^{-1}$. Circles are the functional-interrupt cross section counted from the device's own reset counter normalized by beam-on exposure. Squares are the corruption-event cross section normalized by exposure to clean work cycles. *SRAM_B1*'s functional-interrupt count, the open diamond marked with an asterisk, is not available on a comparable basis, excluding it from the functional-interrupt spread. Builds are ordered by corruption-event cross section. Each bracket shows how far one cross section spreads across builds, and its label gives the ratio of the highest value to the lowest. Each n is the count behind its point. Color identifies firmware, and error bars are 95% exact intervals.

SRAM_Mixed_B200, the highest, together account for 83% of that chi-squared. Whether or not the five builds share one exact rate, that spread is small against the corruption spread measured at the same LET and flux.

The corruption-event rate ranges 1.09×10^{-6} to $1.04 \times 10^{-5} \text{ cm}^2$ across all six builds, a factor of 9.5 coincident with storage placement and clock frequency. The three builds that ran on device B return the three highest rates, so the split between high and low rates is also a split between devices. *SRAM_Mixed_B200* returns the highest rate and *SRAM_B1* the second, level with *FRAM_B1_Throttled*.

The fault site of each event, shown in Fig. 3, is what separates the storage effect from the device. An event counts as reproduced when a single simulated bit flip gives its first corrupted output exactly (Section II-D). Of the 34 events in the SRAM builds, 16 are reproduced by a flip in an input sample, against 1 of the 36 events of *FRAM_B1_Throttled* on the same device (Fisher exact, $p < 10^{-4}$). A match at the output sample cannot be separated from a flip in a register or in transfer, so this comparison uses input samples only. Of the 61 events in the FRAM builds, 84% are not reproduced by any simulated flip, against 26% of the SRAM events. Of the 35 events that are reproduced, 30 match a flip in a stored sample rather than in the arithmetic. Moving working data into FRAM has been reported to eliminate the observed software failures on a related MSP430 [2]. On standalone FRAM parts, single-event effects are located in the peripheral circuitry rather than in the cell array [34, 35]. Under pulsed dose rate and neutrons, a standalone FRAM likewise showed no failures while idle and failed reproducibly only while actively read and written, with

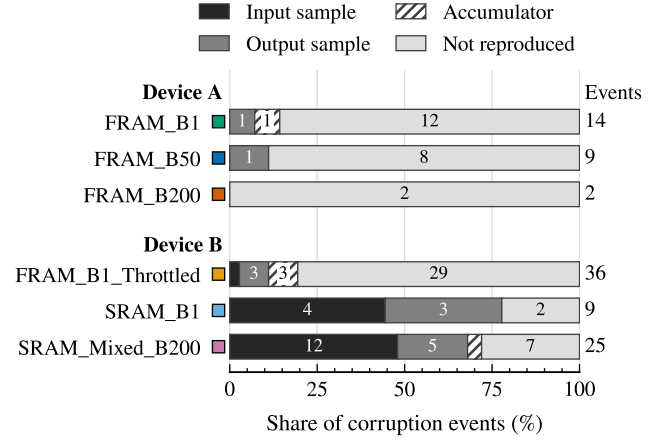


Fig. 3. Fault site of each corruption event at the reference condition, by firmware build and device, located by the single-bit-flip test of Section II-D. Each bar is one build's corruption events, with segment labels giving event counts and the total at right. An event takes the site at which a simulated single flip reproduces its first corrupted output, and no flip at the five simulated sites reproduces the events in the lightest segment. The colored keys match each build's color in Fig. 2.

the corruption confined to the row being transferred [36]. For the 55 FRAM events that are single corrupted outputs rather than persisting runs, the likely mechanism is therefore upsets to the multiplier path or registers in the processor, which are shared with every build. Corruption during communication using the I2C module also cannot be ruled out. The use of unprotected SRAM in this campaign is consistent with an additional corruption mechanism increasing the sensitivity of storage, rather than computation, during the work cycle.

The choice of clock frequency during computation may also move the cross section. In comparison to the three FRAM builds that ran consistently at 16 MHz, the cross section for *FRAM_B1_Throttled* sits noticeably higher. Adding this build to the three pure-FRAM 16 MHz builds rejects a shared rate ($\chi^2 = 18.7$ on three degrees of freedom, $p < 0.001$), with *FRAM_B1_Throttled* carrying 65% of the statistic. Of the four builds, only *FRAM_B1_Throttled* crosses the FRAM wait-state boundary twice per work cycle, as Section II-B describes. It is also the only one of the four that ran on device B, so this campaign cannot separate the clock effect from the device.

Batch size, by contrast, showed no detectable effect on the cross section for these devices and firmware builds. *FRAM_B1*, *FRAM_B50*, and *FRAM_B200* share one storage technology, one clock, and one device, and differ only in batch size, from 1 to 200. Their rates are consistent with a single value ($\chi^2 = 0.6$ on two degrees of freedom, $p = 0.73$), although the test has little power, since *FRAM_B200* alone contributes only 2 events from its limited exposure. The storage and batch-size findings are properties of the firmware rather than of the hardware's underlying response, the same dependence on program behavior reported for other processor benchmarks [1, 37, 38]. The clock finding may also reflect the device.

Unlike the corruption rate, the functional-interrupt rate is close to common across builds, varying by only a factor of 1.4. This commonality is what makes a design calculation possible. Across the six builds at the reference condition,

TABLE IV
QUANTITIES AND TIME BASES USED IN SECTION IV

Quantity	Meaning	Where
Beam-on time	Time the beam was on the device	§II-D
In-cycle time	The part of beam-on time spent inside work cycles	§III-B
Recovery time	The part of beam-on time spent restoring the device after an interruption. Recoveries that paused the beam are counted separately (Section III-A).	§III-A
Detection latency	The host's wait to detect an interrupt, from a reset to its detection, in which a further reset opens no new recovery episode	§IV-B
Dead time	Beam-on time from an interrupt until the device resumes work, in which the host cannot observe it; the detection latency plus the recovery time	§IV-B
Countable exposure	Fluence delivered during work cycles that survive	§IV-B
W	Work-cycle time, from input reception to output transmission	§II-B
θ	Clean-cycle fraction, the share of attempted work cycles that survive; $1 - \theta$ is the share lost	§IV-A
ϕ	Flux, $\text{cm}^{-2}\text{s}^{-1}$	§II
τ	Mean dead time a lost cycle adds beyond its own W , measured here as the recovery time	§IV-B
D	Duty factor, the share of the time spent running cycles or in dead time that yields countable exposure	§IV-B

the share of cycles surviving to deliver a result runs from 0.98 down to 0.51, set almost entirely by work-cycle time. A test engineer cannot predict the application cross section from these results alone, but can predict the rate at which the device stops delivering results.

IV. THE FLUX-SELECTION MODEL

This section applies the results of Section III to form a method for choosing flux and work cycle together. Results from the campaign show that some firmware changes strongly affect device response, such as storage choice and, on one device, clock frequency, while others, like batch size, had no detectable impact. These results support a test-as-you-fly mindset, as the specific hardware exercised affects observed error rates and therefore in-orbit predictions. They also show the tension between a flight application, which may have a long work cycle, and the accelerated flux used for testing, since the work-cycle length sets how many of each build's attempted cycles survived to deliver a result. A balance should therefore be found in which key hardware is kept constant and the work cycle is matched to the flux. Section IV-A sets that balance for the share of cycles that survive, and Section IV-B adds the dead time each lost cycle costs. The quantities this section uses and the time bases normalizing their rates are defined in Tables III and IV.

A. Design Formula and Campaign Validation

Across the six comparison builds, the share of work cycles that survive spans 0.51 to 0.98 at a flux held within an 11% band around $5 \times 10^4 \text{ cm}^{-2}\text{s}^{-1}$. Because the functional-interrupt rate of Section III-B varied by only a factor of 1.4, one cross section for interrupting a running cycle may describe that whole range, leaving work-cycle length to explain the spread. Let θ be the fraction of work cycles of length W that survive. Zimmario et al. [23] write θ as one minus the probability that an interrupt arrives within the cycle, evaluated over the fluence delivered during it. For interruptions arriving at a constant rate, it reduces to the exponential, the survival form used in checkpointed computation [39, 40],

$$\theta = e^{-\sigma_{\text{cyc}}\phi W}. \quad (1)$$

Here σ_{cyc} is the cross section for a lost work cycle and ϕW is the fluence one cycle receives, so a cycle survives or not according to the exposure it accumulates while it runs. Writing the exponent this way holds σ_{cyc} constant in flux, which is the case while each strike acts on its own. In this campaign, W varies from 20 to 814 ms across firmware running one application under heavy ions, and θ is counted directly from lost cycles. The six builds' counts then test whether one σ_{cyc} serves all of them, and the fitted value is compared with σ_{det} , the cross section from the host's own logged detections.

Flux and work-cycle time therefore cannot be chosen independently when functional interrupts limit the observability of the application of interest. A target θ fixes the fluence one cycle may receive, and with it their product,

$$\phi W = -\frac{\ln \theta}{\sigma_{\text{cyc}}}. \quad (2)$$

Every pair of flux and work-cycle time with that product delivers the same share of surviving cycles, so a flux ten times higher is paid for by a work cycle ten times shorter.

Equation (1) was tested against a direct count of surviving cycles from each firmware. Lost cycles are counted as defined in Table III, with a single count per cycle however many detections it carried. At the reference condition, 588 of the 20,459 cycles attempted across the six builds were lost against 787 host detections, the excess being repeat episodes on cycles already lost. The fraction of attempted cycles in which no host-detected interrupt occurred, the clean-cycle fraction, uses neither σ_{cyc} nor W , so comparing it against Equation (1) tests the formula rather than restating it.

A fit on the counted clean-cycle fractions, taking each build at its own flux and work-cycle time, produces a single lost-cycle cross section shared across all six, $\sigma_{\text{cyc}} = 1.56 \times 10^{-5} \text{ cm}^2$, with a 95% interval of 1.44 to $1.69 \times 10^{-5} \text{ cm}^2$. At the nominal matched flux this is 0.78 lost cycles per second of in-cycle time. Fig. 4 plots the directly measured fraction against work-cycle time for each of the six comparison builds, with Equation (1) drawn at the matched flux and the shared cross section. The same curve is drawn at four other fluxes with no further fitting, showing the exponential effect of flux and work cycle on the clean-cycle fraction. Because the exponent of Equation (1) scales with ϕW , pushing flux costs the long-cycle builds far more than the short. At

$2 \times 10^5 \text{ cm}^{-2}\text{s}^{-1}$, the same fit puts the longest cycle at 8% surviving cycles, while the shortest still returns 94%.

Carrying σ_{cyc} measured at one work cycle to another rests on two assumptions, and the direct count of surviving cycles supports both. The first is that one σ_{cyc} serves every build. When each build is given its own value, the counts are described no better than by the shared one (likelihood ratio $G^2 = 6.4$ on five degrees of freedom, $p = 0.27$). The same holds within the three pure-FRAM 16 MHz builds, where only W moves ($p = 0.56$). The second is that a cycle's risk of interruption is constant while it runs, which makes the exponent linear in W . Under a Weibull survival law, $\theta = e^{-(\sigma_{\text{cyc}}\phi W)^\beta}$, a risk that rises as the cycle runs gives $\beta > 1$, one that falls gives $\beta < 1$, and $\beta = 1$ recovers Equation (1). A fit to all six builds with β left free gives $\beta = 0.98$, with a 95% interval of 0.92 to 1.04, and the pure-FRAM builds alone give 0.98 (0.90 to 1.06). Therefore, the agreement across W from 20 to 814 ms is evidence for the form rather than a failure to reject it.

With σ_{cyc} measured in a short pilot, Equation (2) becomes a calibration of flux against work cycle. It maps a target clean-cycle fraction θ to the flux for a fixed work cycle, or to the longest work cycle a fixed flux allows. At this campaign's $\sigma_{\text{cyc}} = 1.56 \times 10^{-5} \text{ cm}^2$, a target $\theta = 0.90$ and the matched flux of $5 \times 10^4 \text{ cm}^{-2}\text{s}^{-1}$, the longest work cycle is 135 ms. Three of the six comparison builds hold a work cycle under that limit, *FRAM_B50* exceeds it by a factor of 1.6, and the two at $B = 200$ exceed it by a factor of roughly 6. The choice of θ is given to the experimenter. The published method chooses flux to maximize the chance of seeing a rarer failure before an interrupt, so θ follows from that choice rather than being set directly [23]. Other guidance, addressed to system-level testing, sets the target differently, recommending a flux that lets at least ten passes through the system's operating modes, its duty cycles, complete between two hard losses of functionality, those needing a power cycle to clear [31].

A cross section measured on beam-on exposure overestimates the surviving share when fed to Equation (1). With the host-detected cross section, $\sigma_{\text{det}} = 8.3 \times 10^{-6} \text{ cm}^2$, or $\sigma_{\text{det}}\phi = 0.42 \text{ s}^{-1}$ against the fitted 0.78 s^{-1} , the equation predicts a higher surviving share than every one of the six builds measured. The host-detected and reset-counter cross sections describe the device's response in this campaign accurately, but their exposure includes the time outside attempted work cycles spent handling interrupts (Section III-B), which increases the normalizing exposure basis without a proportional increase in functional-interrupt counts. The same caution applies to any campaign with dead time in beam.

B. Dead Time in the Flux Choice

Equation (1) counts only in-cycle time, but each lost cycle also costs dead time, the beam-on time from an interrupt until the device resumes work, in which the host cannot observe the device. Including that time shows how much of the beam time yields countable exposure, fluence delivered during cycles that survive to report. It can also change which build makes the best use of the beam. Let τ be the mean dead time a lost cycle

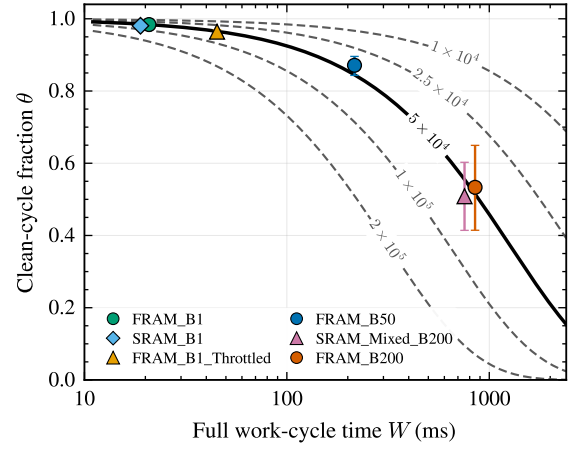


Fig. 4. Measured clean-cycle fraction against work-cycle time for the six comparison builds, with Equation (1) drawn at five fluxes. Points are direct counts of work cycles with no recovery episode, with exact 95% binomial intervals, taken at the matched flux. Curves use the single fitted $\sigma_{\text{cyc}} = 1.56 \times 10^{-5} \text{ cm}^2$ and no other parameter. The solid curve is that fit's own flux, $5 \times 10^4 \text{ cm}^{-2}\text{s}^{-1}$, and the dashed curves are the same equation at other fluxes. Where each curve crosses a target θ , its W is that flux's longest usable work cycle, 135 ms at the matched flux for $\theta = 0.90$. Two pairs of builds fall within 4% in W , so their markers carry a $\pm 4.5\%$ horizontal offset, and every plotted fraction, interval and curve uses the true W .

adds beyond its own W . Charging it to the test gives the duty factor D , the share of the time spent running cycles or in dead time that yields countable exposure. It is the surviving cycle time divided by the sum of cycle time and the dead time expected per cycle, with every attempted cycle, lost or not, counting its full W ,

$$D = \frac{\theta W}{W + (1 - \theta)\tau}. \quad (3)$$

Setting τ to zero returns $D = \theta$, the published relation [23], in which dead time is free. Equation (3) extends that relation to dead time, with τ the sum of the detection latency and the recovery that follow a lost cycle.

A campaign budgets beam time rather than cycles, so a design target can be set on D directly. Inverting Equation (3) gives the clean-cycle fraction that target requires,

$$\theta = \frac{D(W + \tau)}{W + D\tau}, \quad (4)$$

which Equation (2) then turns into a flux.

Equation (3) answers a different question from Equation (1). The clean-cycle fraction asks how many attempted cycles come back clean. The duty factor asks how much of the time a campaign books turns into countable exposure. Fig. 5 shows how far apart the two can be, and the vertical drop from each build's open marker to its filled one is the duty factor lost to measured dead time.

In this campaign, τ is taken as the automatic recovery time, the part of dead time measured directly here. This measurement is the mean duration of a recovery episode times the number of episodes a lost cycle carried. An automatic recovery costs a programmed settle, the delay the host imposes before re-reading the device, plus further recovery attempts when the first does not succeed. Measured per lost cycle, the

cost runs 0.58 to 1.28 s across these builds. With each build's own θ and τ , 34 to 72% of the time spent running cycles and recovering yielded no countable exposure (Fig. 5).

Charging dead time changes which build makes the best use of the beam. *FRAM_B1* and *SRAM_B1* keep the most cycles but rank third and fourth on duty factor, behind *FRAM_B1_Throttled* and *FRAM_B50* (Fig. 5). Measured directly, as the time in cycles that returned a clean result over beam-on time in a call, the countable share follows the same order with no model involved, at 12 to 42%.

The reordering is a physical response of the test, which Equation (3) explains. *FRAM_B1* loses only 1.6% of its 20 ms cycles, but each loss costs 1.17 s of recovery, about sixty cycle-lengths. By the host's own clock, it spent 35% of its beam-on time running cycles and 34% recovering, so a clean-cycle fraction of 0.98 yields a duty factor of only 0.51. *FRAM_B1_Throttled* and *FRAM_B50* lose more of their cycles, 3.6 and 12.9%, but each loss costs them about half as much recovery. They spent only 19 and 15% of their beam-on time recovering, against 43 and 42% running cycles, and keep duty factors of 0.66 and 0.65. Most recoveries took one 0.165 s settle, and most of the recovery time went to those whose first attempt failed. That happened in 15 and 22% of recoveries for *FRAM_B1* and *SRAM_B1*, against 6 to 11% for the rest. In the equation, the dead time charged per unit of cycle time is $(1 - \theta)\tau/W$, and while few cycles are lost, $1 - \theta$ grows in proportion to W . The cycle length then drops out, and τ sets how much test time is lost. For a build like *FRAM_B1*, a shorter cycle gains little, while less dead time gains directly.

With recovery as the only dead time in τ , D is an upper bound on the directly measured countable share, because it leaves out the host's wait to detect an interrupt and the re-running of cycles (Section III-B). That wait, the detection latency, is dead time too. It is not logged, but the host's timing of each interrupted iteration puts it at 0.7 to 1.1 s per lost cycle at $B \leq 50$, comparable to the recovery itself. Charging it brings D within 0.02 to 0.05 of the measured share, for example 0.39 against 0.37 for *FRAM_B1*. At $B = 200$, the gap comes mostly from re-running cycles, which took about 40% of beam-on time (Section III-B). Re-running is not dead time, since the host observes the device throughout, but a re-run does not begin from a verified state and is not counted as a work cycle, so it lowers the measured share without entering τ .

Most of this dead time is set by the host, which is where an experimenter can gain test time. Ensuring recovery succeeds the first time, or shortening the resynchronization timeout that each failed attempt waits out, gains more than a shorter settle. A second of dead time costs D the same whatever its source, so at $B \leq 50$, detecting interrupts sooner gains as much as recovering faster. Pausing the beam to recover does not escape the charge, which moves from beam-on time to facility time.

Dead time also raises a question of counting. If a second strike lands during it, the device resets again but no new recovery episode opens. This is the dead-time counting loss known from radiation detectors [41]. On a system recovered by an external watchdog, a counted cross section fell by a factor

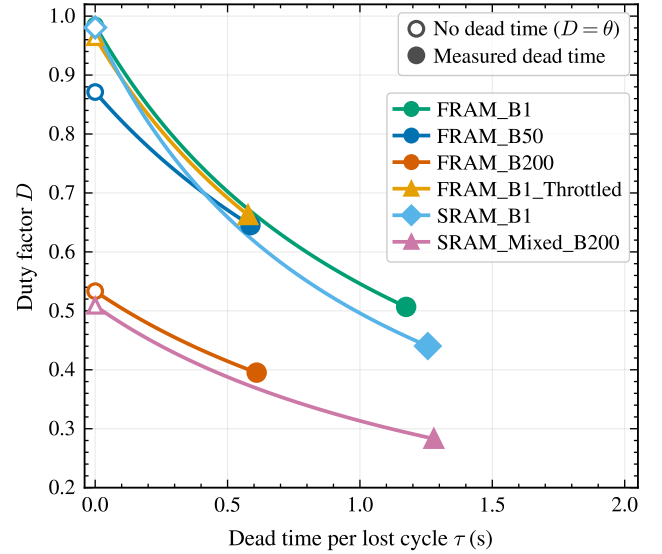


Fig. 5. Duty factor, Equation (3), against dead time per lost cycle for each comparison build, using its measured θ and W . Open marker: no dead time, where $D = \theta$. Filled marker: the measured dead time, its recovery part. Reading along any vertical line gives the order the builds would hold at a common dead time.

of 3.76 as flux rose by a factor of 10.5 [15]. Here the device's reset counter records these hidden resets, but it also records repeated resets that follow a single event, so the true interrupt rate lies between the host's count and the counter's. The two can be separated by rate, because only the hidden share of interrupts grows as flux rises. At the reference condition, 11 to 13% of the interrupts in the four FRAM builds, whose reset counters read cleanly, were hidden this way. Their true interrupt rate therefore likely lies 12 to 16% above the host-detected rate and 7 to 10% below the reset-counter rate. Most of the loss arose during the detection latency, and only 2 to 3 percentage points arose during recovery. Pausing the beam or closing a shutter on detection [14, 42] would therefore remove only those 2 to 3 points. The detection latency would remain, set here by the host's own wait and timeouts, which also took up to 29% of beam-on time (Section III-B). For a test that quantifies risk to the operation, hidden strikes bias a host-detected interrupt rate low, and with it a projected mission interrupt rate. They do not bias the clean-cycle fraction or the corruption-event rate, since a hidden strike lands in dead time, after its cycle is already lost.

V. LIMITATIONS AND FUTURE WORK

Several limits bound what this campaign supports. The firmware comparison rests on two devices, and both SRAM builds and *FRAM_B1_Throttled* ran on device B. Storage and clock differences therefore cannot be fully separated from a device difference, beyond the within-device contrast of Section III-B. Previous total-ionizing-dose testing of this device population found a wide range in time to first failure across parts [43]. Although that work concerned a different radiation effect, part-to-part variation in this population is documented and cannot be excluded here. Accumulated dose over the campaign reached 17 krad(SiO₂) on the most exposed

device, which carried no comparison build. Devices A and B ended at 7.8 and 16.7 krad(SiO₂), well below the doses at which parts of this population first failed under total ionizing dose.

Additional limits concern how interrupts were counted. The hidden-strike estimate of Section IV-B uses rate to separate hidden strikes from the repeated resets a single event can cause, but it cannot say which kind any one reset was. Allowing for its uncertainty, the hidden share could lie anywhere from 6 to 17%. A sweep of flux at one fixed LET, with repeat runs at each flux point, would test the counting loss directly [18, 20]. The reset counter also credits a gap of n as n interrupts, although part of that credit is likely repeated resets after a single event. It recorded 13 reset loops, none at the reference condition, and crediting each once would lower the sweep of Section III-A from 4,448 to 3,951 interrupts. Crediting every multi-reset gap as one interrupt would leave the five comparison builds consistent with one shared rate ($p = 0.64$), so the rejection in Section III-B depends on how such gaps are credited. The finding that the functional-interrupt rate varies far less than the corruption rate holds either way, since the spread narrows from a factor of 1.4 to 1.2.

VI. CONCLUSIONS

A heavy-ion campaign on the MSP430FR6989 was presented in which six firmware builds ran a shared task at a shared LET and flux. The device's own reset path, with an automatic recovery under beam, resolved most functional interrupts, and for the build swept across LET it delivered 18 times the exposure that pausing the beam at every detection would have allowed. Across the six builds, the corruption-event rate spread by a factor of 9.5, against a factor of 1.4 in the functional-interrupt rate over the five builds directly comparable on that count. Storage placement is the leading candidate for the larger spread, supported by an input-buffer signature that appears in the SRAM builds and not in a FRAM build on the same device. The one build that slowed its clock during computation also rose, although it shared a device with both SRAM builds. Batch size alone showed no detectable effect in a test with little power, which makes it a candidate for matching a test's work cycle to flight conditions. The spread in corruption-event cross section between these firmware versions emphasizes the need for mission-representative test design for complex devices, as the choice in exercised hardware affects the rates that will be projected to a mission. However, the lengthy work cycle that a true mission-representative test design might have clashes with the accelerated flux used in a heavy-ion test. The share of work cycles surviving to deliver a result fell from 0.98 to 0.51 across the six builds at a matched LET and flux, tracking work-cycle time, so the work cycle sets how many attempted cycles are lost. Therefore, flux, the device's susceptibility to functional interrupts, and the work cycle of the test must be chosen together.

A flux-selection method developed to observe a rare failure mode before the work cycle is interrupted [23] was put to a new use here, calibrating flux against work cycle. Tested on six firmware builds with work-cycle times from 20 to 814 ms,

its survival relation holds with a single cross section, and a Weibull shape fitted freely returns the constant-rate form, showing that the risk of interrupt neither falls nor rises as a work cycle proceeds for these builds. A cross section measured in a short pilot therefore sets the flux for an application's work cycle, or the longest work cycle a chosen flux allows, at a target share of surviving cycles.

These flux-selection methods are further extended to represent the effective use of beam-facility time by incorporating dead time, the time under beam in which the host cannot observe the device. Of the time this campaign spent recovering from functional interrupts, waiting to detect them, and re-running the cycles they cut short, the first two are dead time, and only the recovery was measured directly. Counting recovery as the only dead time, 34 to 72% of the time spent running cycles and recovering produced no countable exposure, depending on the firmware build. That consideration reorders the builds, and those keeping the highest fraction of clean cycles are not those delivering the most exposure to the application of interest. A test can therefore be well designed on the clean-cycle fraction alone and still spend most of its beam time producing no countable exposure. The reordering comes from dead time rather than cycle length. A build already keeping most of its cycles gains more from reducing its dead time, by recovering reliably or detecting interrupts sooner, than from shortening the cycle it runs.

A separate risk attaches to dead time itself: a second strike during it can reset the device again without the host detecting it. At the reference condition, 11 to 13% of interrupts were hidden this way, which lowers the host-detected interrupt rate but not the clean-cycle fraction or the corruption-event rate. Most of that loss arose during the detection latency, so pausing the beam or closing a shutter on detection does not remove the majority.

Three measured inputs and one choice set a dynamic test's flux and work cycle: the lost-cycle cross section from a pilot, the recovery time and the detection latency that make up the dead time per lost cycle, and the targeted share of surviving cycles.

REFERENCES

- [1] H. Quinn, "Challenges in testing complex systems," *IEEE Trans. Nucl. Sci.*, vol. 61, no. 2, pp. 766–786, Apr. 2014.
- [2] H. Quinn, T. Fairbanks, J. L. Tripp, G. Duran, and B. Lopez, "Single-event effects in low-cost, low-power microprocessors," in *Proc. IEEE Radiat. Effects Data Workshop (REDW)*, Paris, France, Jul. 14–18, 2014, pp. 246–254.
- [3] R. Koga, S. Penzin, K. Crawford, and W. Crain, "Single event functional interrupt (SEFI) sensitivity in microcircuits," in *Proc. RADECS 1997*, Cannes, France, Sep. 15–19, 1997, pp. 311–318.
- [4] P. V. Nekrasov, A. B. Karakozov, D. V. Bobrovskiy, and V. A. Marfin, "Investigation of single event functional interrupts in microcontroller with PIC17 architecture," in *Proc. RADECS 2015*, Moscow, Russia, Sep. 14–18, 2015, pp. 139–142.
- [5] S. Esquer et al., "Fault propagation in microprocessors with configurable cache memory," in *Proc. RADECS 2022*, Venice, Italy, Oct. 3–7, 2022, pp. 249–256.
- [6] S. Houssany, N. Guibaud, F. Miller, T. Cheviron, and T. Colladant, "Experimental test approach for SEFI categorization in microprocessors," in *Proc. RADECS 2021*, Vienna, Austria, Sep. 13–17, 2021, pp. 68–74.

- [7] L. Noizette, F. Miller, T. Colladant, Y. Helen, and R. Leveugle, "Understanding the link between complex digital devices soft error rate and the running software," *IEEE Trans. Nucl. Sci.*, vol. 70, no. 8, pp. 1747–1754, Aug. 2023.
- [8] R. Velazco, S. Karoui, and T. Chapuis, "SEU testing of 32-bit microprocessors for space application," in *Proc. IEEE Radiat. Effects Data Workshop (REDW)*, New Orleans, LA, USA, Jul. 13–17, 1992, pp. 16–20.
- [9] H. Quinn, K. Gnawali, and S. Tragoudas, "Updates on testing microprocessors effectively," *IEEE Trans. Nucl. Sci.*, vol. 68, no. 5, pp. 842–849, May 2021.
- [10] S. Esquer, J. K. Mee, H. M. Quinn, B. D. Sierawski, and R. D. Schrimpf, "Characterization of single event functional interrupts in COTS and rad-hard ARM microcontrollers," in *Proc. IEEE Space Comput. Conf. (SCC)*, Mountain View, CA, USA, Jul. 15–19, 2024, pp. 28–39.
- [11] S. Esquer, B. D. Sierawski, A. F. Witulski, R. D. Schrimpf, G. Karsai, and M. Turowski, "Single event functional interrupt (SEFI) sensitivities of a multicore microprocessor," in *Proc. IEEE Aerosp. Conf.*, Big Sky, MT, USA, Mar. 2–9, 2024, pp. 4879–4889.
- [12] P. Rech, L. Pilla, P. Navaux, and L. Carro, "Impact of GPUs parallelism management on safety-critical and HPC applications reliability," in *Proc. 44th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Atlanta, GA, USA, Jun. 23–26, 2014, pp. 455–466.
- [13] M. Bohman, B. James, M. J. Wirthlin, H. Quinn, and J. Goeders, "Microcontroller compiler-assisted software fault tolerance," *IEEE Trans. Nucl. Sci.*, vol. 66, no. 1, pp. 223–232, Jan. 2019.
- [14] Texas Instruments Incorporated, "Single-event effects test report of the MSP430FR5969-SP," Texas Instruments Incorporated, Tech. Rep. SLSEC56, 2019.
- [15] A. Zimmaro et al., "Testing and validation methodology for a radiation monitoring system for electronics in particle accelerators," *IEEE Trans. Nucl. Sci.*, vol. 69, no. 7, pp. 1642–1650, Jul. 2022.
- [16] L. Frias-Dominguez et al., "Radiation reliability of system reboots in commercial off-the-shelf SoC," *IEEE Trans. Nucl. Sci.*, vol. 72, no. 8, pp. 2662–2670, Aug. 2025.
- [17] S. Buchner, P. W. Marshall, S. Kniffin, and K. A. LaBel, "Proton test guideline development—lessons learned," NASA Goddard Space Flight Center, Electronic Radiation Characterization Project, Tech. Rep., Jan. 2002.
- [18] M. D. Berg, K. A. LaBel, H. Kim, M. Friendlich, A. Phan, and C. Perez, "A comprehensive methodology for complex field programmable gate array single event effects test and evaluation," *IEEE Trans. Nucl. Sci.*, vol. 56, no. 2, pp. 366–374, Apr. 2009.
- [19] J. A. Zoutendyk, H. R. Schwartz, R. K. Watson, Z. Hasnain, and L. R. Nevill, "Single-event upset (SEU) in a DRAM with on-chip error correction," *IEEE Trans. Nucl. Sci.*, vol. NS-34, no. 6, pp. 1310–1315, Dec. 1987.
- [20] G. R. Allen, L. Edmonds, C. W. Tseng, G. Swift, and C. Carmichael, "Single-event upset (SEU) results of embedded error detect and correct enabled block random access memory (BlockRAM) within the Xilinx XQR5VFX130," *IEEE Trans. Nucl. Sci.*, vol. 57, no. 6, pp. 3426–3431, Dec. 2010.
- [21] G. Allen et al., "Single event test methodologies and system error rate analysis for triple modular redundant field programmable gate arrays," *IEEE Trans. Nucl. Sci.*, vol. 58, no. 3, pp. 1040–1046, Jun. 2011.
- [22] European Space Components Coordination, "Single event effects test method and guidelines," European Space Components Coordination, Tech. Rep. ESCC Basic Specification No. 25100, Issue 2, Oct. 2014.
- [23] A. Zimmaro et al., "Radiation test flux selection methodology to optimize SEE observability on systems with different operating modes," in *Proc. RADECS 2022*, Venice, Italy, Oct. 3–7, 2022, pp. 229–236.
- [24] Facility for Rare Isotope Beams, Michigan State University, "K500 single event effects facility calibration procedures and results," Facility for Rare Isotope Beams, Michigan State University, Tech. Rep. FRIB-M42200-RC-010012-R002, 2026.
- [25] Texas Instruments Incorporated, "MSP430FR6989 mixed-signal microcontroller datasheet," Texas Instruments Incorporated, Tech. Rep. SLAS824A, Apr. 2016.
- [26] Texas Instruments Incorporated, "MSP430FR58xx, MSP430FR59xx, and MSP430FR6xx family user's guide," Texas Instruments Incorporated, Tech. Rep. SLAU367P, Apr. 2020.
- [27] Texas Instruments Incorporated, "MSP DSP library (DSPLib): Getting started," Texas Instruments Incorporated, Tech. Rep., version 1.30.00.02, 2015.
- [28] JEDEC Solid State Technology Association, "Test procedures for the measurement of single-event effects in semiconductor devices from heavy ion irradiation," JEDEC Solid State Technology Association, Tech. Rep. JESD57A, Nov. 2017.
- [29] J. D. Black, P. E. Dodd, and K. M. Warren, "Physics of multiple-node charge collection and impacts on single-event characterization and soft error rate prediction," *IEEE Trans. Nucl. Sci.*, vol. 60, no. 3, pp. 1836–1851, Jun. 2013.
- [30] H. M. Quinn, D. A. Black, W. H. Robinson, and S. P. Buchner, "Fault simulation and emulation tools to augment radiation-hardness assurance testing," *IEEE Trans. Nucl. Sci.*, vol. 60, no. 3, pp. 2119–2142, Jun. 2013.
- [31] A. Coronetti et al., "Radiation hardness assurance through system-level testing: Risk acceptance, facility requirements, test methodology, and data exploitation," *IEEE Trans. Nucl. Sci.*, vol. 68, no. 5, pp. 958–969, May 2021.
- [32] F. Garwood, "Fiducial limits for the Poisson distribution," *Biometrika*, vol. 28, no. 3/4, pp. 437–442, Dec. 1936.
- [33] W. E. Ricker, "The concept of confidence or fiducial limits applied to the Poisson frequency distribution," *J. Am. Stat. Assoc.*, vol. 32, no. 198, pp. 349–356, Jun. 1937.
- [34] A. L. Bosser et al., "Single-event effects in the peripheral circuitry of a commercial ferroelectric random access memory," *IEEE Trans. Nucl. Sci.*, vol. 65, no. 8, pp. 1708–1714, Aug. 2018.
- [35] A. Ju et al., "Failure analysis of commercial ferroelectric random access memory for single event effect," *IEEE Trans. Nucl. Sci.*, vol. 69, no. 4, pp. 890–899, Apr. 2022.
- [36] N. Harris, W. Stirr, D. Black, J. Black, M. Wirthlin, and J. Goeders, "Dynamic testing of a commercial FRAM device under gamma ray dose and neutron beam," *IEEE Trans. Nucl. Sci.*, vol. 71, no. 8, pp. 1896–1905, Aug. 2024.
- [37] H. Quinn et al., "Using benchmarks for radiation testing of microprocessors and FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 62, no. 6, pp. 2547–2554, Dec. 2015.
- [38] W. Stirr, E. Poff, J. Smith, J. Goeders, and M. Wirthlin, "Comparison of neutron radiation testing approaches for a complex SoC," *IEEE Trans. Nucl. Sci.*, vol. 70, no. 4, pp. 505–514, Apr. 2023.
- [39] J. W. Young, "A first order approximation to the optimum checkpoint interval," *Commun. ACM*, vol. 17, no. 9, pp. 530–531, Sep. 1974.
- [40] J. T. Daly, "A higher order estimate of the optimum checkpoint interval for restart dumps," *Future Gener. Comput. Syst.*, vol. 22, no. 3, pp. 303–312, Feb. 2006.
- [41] G. F. Knoll, *Radiation Detection and Measurement*, 4th ed. Hoboken, NJ, USA: John Wiley & Sons, 2010.
- [42] T. Wilcox, M. Campola, M. Kadari, and G. Nadendla, "Single event effect testing of the Analog Devices ADV212," NASA Goddard Space Flight Center, Radiation Effects and Analysis Group, Tech. Rep. NASA/NEPP, NTRS 20170007439, Jun. 2017.
- [43] I. Hudson, H. Hunnicutt, and T. D. Loveless, "Individualized screening and in-situ health monitoring for TID-induced effects in a complex microcontroller," submitted for publication, 2026.